# Capítulo 11. Aumenta tu productividad

Este trabajo se ha traducido utilizando IA. Agradecemos tus opiniones y comentarios: translation-feedback@oreilly.com

> *Céntrate en ser productivo en lugar de estar ocupado.*
>
> —Tim Ferriss, empresario, inversor y autor estadounidense

Es posible que hayas oído hablar del llamado desarrollador 10× de , el ingeniero de software que se considera diez veces más productivo que un desarrollador medio. Si realmente existen es discutible, pero es innegable que *hay* personas que realizan las tareas más rápido y de forma más eficiente que otras. Para convertirte en uno de ellos, busca siempre formas de mejorar, empezando por tu kit de herramientas de desarrollo, el editor de código, la línea de comandos y todas esas pequeñas utilidades que los ingenieros sénior parecen conocer. Este capítulo te ayudará a personalizar tu entorno de desarrollo y a convertirte en una ingeniería de software más productiva. Aprenderás a controlar

tu editor, a navegar con facilidad por la línea de comandos y por qué dominar los atajos de teclado es una superpotencia.

# Optimización de tu entorno de desarrollo

Los primeros desarrolladores tuvieron que conformarse con herramientas bastante rudimentarias. Por suerte, tú dispones de una gran variedad de editores increíbles y completos, como IntelliJ IDEA y VS Code. Pero instalar una herramienta no es el final del camino, sino solo el principio.

---

**ADVERTENCIA**

Siéntete libre de explorar diferentes configuraciones, pero ten cuidado de no perder demasiado tiempo en ello. Si no tienes cuidado, podrías encontrarte jugando con las fuentes y los temas cuando realmente deberías estar trabajando. Es aconsejable limitar el tiempo de tus experimentos. Si se lleva demasiado lejos, las personalizaciones también pueden hacer que sea casi imposible trabajar en pareja o de forma eficaz con otras personas. Nate intentó una vez trabajar en pareja con un desarrollador que solo podía trabajar dentro de Vim. Utilizando un teclado especializado. Con la configuración Dvorak. No salió bien.

---

## Conoce tus herramientas de desarrollo

En muchas profesiones, tú eres responsable de tus herramientas. En una cocina profesional no hay un bloque de cuchillos común; los chefs

son responsables de traer sus propios cuchillos al trabajo. Los mecánicos suelen tener su propia caja de herramientas con sus herramientas preferidas organizadas según sus estándares. En muchas profesiones, la compra de las herramientas necesarias es el costo de entrada al oficio. Aunque tu empresa pueda cubrir el costo de las licencias de software, tú sigues siendo responsable de tu kit de herramientas, y no debes tener miedo de hacerlo tuyo.

Como ingeniero de software, debes hacerte cargo de tus herramientas de desarrollo. Los IDE modernos te animan a crear el entorno perfecto mediante la selección del conjunto adecuado de complementos. Debes aprender a manejar los instrumentos de tu oficio. Eso no significa que tengas que conocer todos los rincones y recovecos y todos los comandos o elementos del menú más oscuros. Pero debes conocer como la palma de tu mano las cosas que utilizas día tras día.

Tu objetivo es mantenerte en el estado de flujo « », tal y como lo describe Mihaly Csikszentmihalyi en *Flow: The Psychology of Optimal Experience* (Harper & Row). *Los estados de flujo* son períodos de profunda concentración en los que estás completamente absorto en las tareas que tienes entre manos. El código parecerá aparecer por arte de magia en la pantalla. Detenerte cada pocos minutos para buscar un comando o alcanzar el trackpad cuando podrías haber utilizado un atajo de teclado puede interrumpir el flujo y romper tu concentración. (Más adelante en este capítulo encontrarás más información sobre los atajos de teclado).

> *El código fluye del cerebro al IDE y, antes de que te des cuenta (ya que te sabes de memoria las combinaciones de teclas de tu IDE), estás creando/probando/ejecutando.*
>
> —Mark Heckler, autor de *Spring Boot: Up and Running*

Tómate tu tiempo para aprender más sobre tu editor y tu herramienta de gestión de código fuente preferida. Es mejor que no seas tú quien provoque que el equipo pierda tres semanas de trabajo por forzar imprudentemente un push a la rama principal.

Los IDE modernos cuentan con enormes capacidades, lo que hace casi imposible descubrir todas sus funcionalidades. Un enfoque más eficaz es comprometerse a aprender una nueva función cada semana. Por ejemplo, como hemos mencionado en el capítulo 2, JetBrains IntelliJ IDEA ofrece un útil «consejo del día» al iniciarse. Es una forma estupenda de descubrir atajos, herramientas, refactorización, depuración y complementos que quizá no hubieras encontrado por tu cuenta. Busca funciones similares en otros IDE y aprende algo nuevo.

## Construye tu propio sable láser

Cuando te pones al volante de un coche, ajustas el asiento, el volante y los espejos a tu medida; utilizar los ajustes predeterminados de fábrica no sería cómodo (ni seguro). Del mismo modo, debes personalizar tu entorno de desarrollo para que se adapte *a tus* necesidades específicas. Tus herramientas tienen una serie de ajustes, por lo que te debes a ti mismo dedicar algo de tiempo a optimizarlas según tus preferencias.

Piensa en ello como si estuvieras construyendo tu propio sable láser: quieres que tus herramientas se adapten a tus manos, a tu personalidad y a tus necesidades. No tengas miedo de personalizar tu entorno.

Pasarás horas y horas mirando el código en tu editor. Aunque la fuente predeterminada puede ser estupenda, tus ojos te agradecerán que dediques un tiempo a explorar las opciones disponibles. Una fuente diferente no solo te puede parecer más atractiva, sino que también te ofrece otra forma de personalizar tu entorno para reflejar tu personalidad.

No faltan excelentes opciones monoespaciadas entre las que elegir. Una rápida búsqueda en Internet te mostrará muchas listas de las mejores fuentes monoespaciadas para codificar y programar. Haz una prueba: prueba una durante un día o dos y ve qué te parece. No tengas miedo de probar otra. Tampoco hay nada mágico en las fuentes de 11 o 12 puntos; si encuentras otra que se adapte mejor a tu vista, ¡cámbiala!

La mayoría de los editores tienen innumerables temas que alteran la apariencia de casi todo en la herramienta. A muchos desarrolladores les encanta el modo oscuro, ya que mirar una pantalla muy brillante todo el día puede ser perjudicial para la vista. Dicho esto, los temas oscuros pueden adaptarse o no a tu vista, así que prueba varios hasta que encuentres uno que te guste.[1] Algunos sistemas operativos ajustan automáticamente el brillo de la pantalla a tu entorno y/o a la hora del día, lo que, de nuevo, puede reducir la fatiga visual. Independientemente de tus preferencias, debes modificar los temas y encontrar el que mejor se adapte a ti.

No te limites al editor. Es probable que tu sistema operativo se pueda ajustar a tus necesidades exactas; es posible que la configuración predeterminada no sea la adecuada para ti, así que modifícala según tus preferencias. Prueba a colocar el dock en un lugar diferente, cambia el aumento o actualiza la imagen de fondo por otra que te guste más.

Existe todo un ecosistema de utilidades gratuitas o de bajo coste para desbloquear cualquier número de funciones con las que personalizar tu entorno de trabajo. Desde utilidades para el portapapeles hasta un control más detallado del sonido, pasando por aplicaciones que te permiten reorganizar la barra de menús, es muy probable que con solo una breve instalación puedas eliminar algunas molestias de tu día a día. Si ves algo que no reconoces en la barra de menús de un compañero, pregúntale por ello; ese descubrimiento podría ahorrarte horas al mes con solo preguntar. Una búsqueda en Internet te llevará a un sinfín de herramientas que puedes añadir a tu arsenal.

No olvides que también puedes personalizar tu entorno físico e . Ajusta tu silla y tu monitor a una altura cómoda que mantenga tu cuerpo en una posición neutra. Algunas personas prefieren un escritorio de pie, y hay muchas opciones excelentes que te permiten cambiar de posición a lo largo del día.[2] Si tu empresa tiene un departamento de ergonomía, pídele que realice una evaluación de tu espacio de trabajo y no tengas miedo de solicitar una silla mejor o un escritorio ajustable. Si trabajas desde casa, es posible que tu empresa te ofrezca una asignación para comprar equipo, pero si no es así, debes invertir en un escritorio y una silla de calidad; tu espalda te lo agradecerá.[3]

La fatiga visual es algo real, y mirar pantallas todo el día puede ser problemático. Toma descansos a lo largo del día para enfocar la vista a distancias distintas a la de tu monitor. Si usas gafas, asegúrate de que tu optometrista sepa que pasas mucho tiempo mirando pantallas. Desde diferentes recubrimientos hasta lentes especiales, hay muchas opciones que pueden aliviar la fatiga visual. Algunos desarrolladores incluso tienen un juego de monturas separado con una prescripción modificada para usar cuando programan.

---

**COMPRA UN TECLADO MEJOR**

Hablando de teclados, no tengas miedo de comprar uno mejor. Es probable que el que haya comprado tu empresa al por mayor no sea el adecuado para ti. Un teclado ergonómico dividido, como el ErgoDox EZ, infinitamente personalizable, puede marcar una gran diferencia en tu vida diaria. Con la posibilidad de elegir el interruptor de teclas que más te guste, debes usar el que mejor te funcione. Muchas empresas te reembolsarán parte o la totalidad del coste si lo solicitas.[4]

---

## Aprovecha el poder de la línea de comandos

Antes de la llegada e e de los sistemas de ventanas y los entornos informáticos «lo que ves es lo que obtienes», los ordenadores eran sistemas puramente basados en texto que requerían comandos arcanos, crípticos y, a menudo, implacables. Debes aprovechar sin duda los sistemas informáticos modernos de hoy en día, pero como ingeniero de softwa-

re, también debes sentirte cómodo con la interfaz de línea de comandos (CLI).

Aunque la línea de comandos puede resultar intimidante al principio, aprovechar la CLI se convierte rápidamente en un superpoder y un multiplicador de fuerzas. Muchas tareas comunes se pueden realizar de forma mucho más eficiente con solo pulsar unas teclas. Dedica unos minutos a la semana a probar nuevos comandos.

---

**CONSEJO**

¡No olvides que el manual está integrado en la línea de comandos! Si no estás seguro de lo que hace algo, solo tienes que buscarlo con el comando « `man` ».

---

Los comandos de shell son tus aliados. Probablemente estés familiarizado con cosas como `cd`, `pwd`, `ls` y `mkdir`, pero hay muchos más a tu disposición. Desde la línea de comandos, puedes iniciar fácilmente una aplicación mediante `open`. Comandos como `cat`, `cut`, `grep` y `pbcopy` pueden ahorrarte un tiempo valioso. Por supuesto, todos ellos se pueden combinar utilizando el comando pipe (|). Pipe te permite encadenar herramientas sencillas y de un solo uso para obtener resultados más complejos. Por ejemplo, ¿alguna vez has querido obtener la URL de un repositorio Git? `cat .git/config | grep url | cut -f2 -d= | pbcopy` te ayudará.

¿Alguna vez has querido que tu ordenador te hablara? Prueba esto:

```
say I can’t let you do that Nate.
```

También puedes utilizar otras herramientas CLI de . ¿Alguna vez has querido `cat` pero con resaltado de sintaxis? Prueba `bat` . ¿Te gustaría tener diferencias agradables? Echa un vistazo a `diff so fancy` . ¿Pasas mucho tiempo mirando JSON? Considera la posibilidad de añadir `fx` a tu conjunto de herramientas. ¿Te gustaría tener `ls` pero con mejores colores? Prueba a instalar `exa` .

Muchos desarrolladores se toman el tiempo de crear alias de shell para las cosas que hacen a menudo en la línea de comandos. Con solo añadir una o dos líneas al archivo de configuración de tu shell (por ejemplo, `~/.bashrc` para bash o `~/.zshrc` para Z shell), puedes ahorrarte innumerables pulsaciones de teclas. Son muy útiles hasta que trabajas en una máquina que no tiene tus modificaciones específicas, así que úsalas con prudencia. Aprovecha estas funciones para las tareas que realizas con frecuencia o que requieren mucho tiempo para escribir. Por ejemplo:

```
alias k=kubectl

alias d='docker'

alias dc='docker compose'
```

No te olvides tampoco del historial del shell. El historial del shell es uno de los aspectos más subestimados del trabajo en la línea de comandos, ya que te permite desplazarte hacia arriba por tus comandos recientes y pulsar la tecla Tab para autocompletar cualquier cosa que hayas hecho recientemente. El comando `history` te dará una lista de

lo que has introducido anteriormente, y Ctrl-R te permitirá realizar una búsqueda inversa en tu historial de comandos.

Todo eso se puede ajustar: puedes aumentar el tamaño del historial o incluso ignorar ciertos comandos. Hay un sinfín de trucos del shell que puedes aprovechar. Añade un buscador difuso a tu configuración y es muy probable que nunca vuelvas a escribir un comando más de una vez.

Por último, no te limites al shell predeterminado de tu sistema operativo. Hay muchas opciones excelentes disponibles, como Oh My Zsh. Además de muchas funciones integradas útiles, Oh My Zsh tiene miles de complementos, temas y ayudantes.

## Aprovecha el poder de los atajos de teclado

Los sistemas operativos modernos aprovechan el poder del puntero, ya sea un ratón, un trackpad o tu propio dedo. Si bien eso sin duda ha hecho que las computadoras sean mucho más accesibles, tu objetivo debe ser mantener las manos en la fila central del teclado.[5] Aprende los atajos de teclado. No solo son más rápidos y eficientes, sino que también te ahorran el tiempo de mover constantemente la mano al ratón o al trackpad. El uso de atajos también reduce la tensión en las muñecas y las manos.

CONSEJO

Si no escribes sin mirar el teclado, deberías aprender a hacerlo. En serio. Los teléfonos inteligentes han enseñado a toda una generación a escribir con los pulgares, y aunque eso es útil para conversar con tus amigos, como desarrollador debes usar todos los dedos para escribir código. Escribir sin mirar el teclado es más rápido y más fácil para las manos, los ojos y el cuello. Es probable que no hayas aprendido a escribir en la escuela, pero hay muchas opciones en línea que han convertido el proceso en un juego. Aprende a escribir sin mirar el teclado; nos lo agradecerás, te lo prometemos.

Dominar los atajos de teclado marca la diferencia entre los principiantes y los profesionales experimentados, y rápidamente se convierten en un multiplicador de fuerzas. Aunque debes aprender los atajos de teclado de tu editor preferido, también debes aprender los de tu sistema operativo.

Algunos atajos están tan interiorizados que tus manos los ejecutan sin que te des cuenta. La mayoría de los usuarios de ordenadores están familiarizados con opciones como Cmd-C y Cmd-V, pero hay muchas más a tu disposición. ¿Cuántas ventanas del navegador tienes abiertas en este momento? Si quieres ver todas las ventanas de la aplicación en primer plano, Ctrl-flecha abajo te ayudará. ¿Necesitas ocultar la aplicación en primer plano? Cmd-H te ayudará. En IntelliJ, si seleccionas dos archivos y pulsas Cmd-D, verás las diferencias entre ambos. Esto también funciona con archivos JAR y ZIP.

Muchas aplicaciones y sistemas operativos funcionan con las combinaciones de teclas de Emacs, muchas de las cuales son increíblemente

útiles, especialmente cuando se trata de navegar por el texto (como los archivos de código, por ejemplo). Para avanzar o retroceder un solo carácter, pulsa Ctrl-F y Ctrl-B. También puedes avanzar o retroceder una palabra completa utilizando Meta-F y Meta-B, respectivamente.[6] Puedes desplazarte al principio o al final de una línea con Ctrl-E y Ctrl-A. Dedicar unos minutos a aprender algunas combinaciones de teclas de Emacs puede ahorrarte mucho tiempo.

---

**EMACS FRENTE A VI**

Si realmente quieres iniciar una batalla real en una sala de proyectos, debate sobre Emacs versus vi. En serio, hace que la discusión sobre las tabulaciones versus los espacios parezca pintoresca en comparación. Basta con decir que «Emacs es un gran sistema operativo, al que solo le falta un editor decente», mientras que «Vim es un gran editor, al que le falta por completo un gestor de ventanas decente». Quedas avisado.

Dicho esto, aprender los conceptos básicos de vi puede ser increíblemente útil. En algunos casos, puede que sea el único editor que tengas disponible. Y, de nuevo, aprender las combinaciones de teclas de Emacs puede convertirte en un experto en la navegación por archivos.

---

Incluso la combinación de teclas más utilizada, Ctrl-C, Ctrl-V, se puede optimizar. Considera la posibilidad de utilizar una herramienta como Pastebot, como se muestra en la Figura 11-1, que te da acceso a tu historial de pegado. ¿Cuántas veces has necesitado copiar cinco cosas de una aplicación a otra? ¿Te ha gustado tener que cambiar de ventana?

Probablemente no. Una herramienta como Pastebot te permite agrupar tus copias y pegados, lo que puede ahorrarte mucho tiempo. Pastebot también te permite volver atrás en tu historial (ten cuidado con las contraseñas y otra información confidencial) para volver a pegar algo que hayas copiado anteriormente ese mismo día.

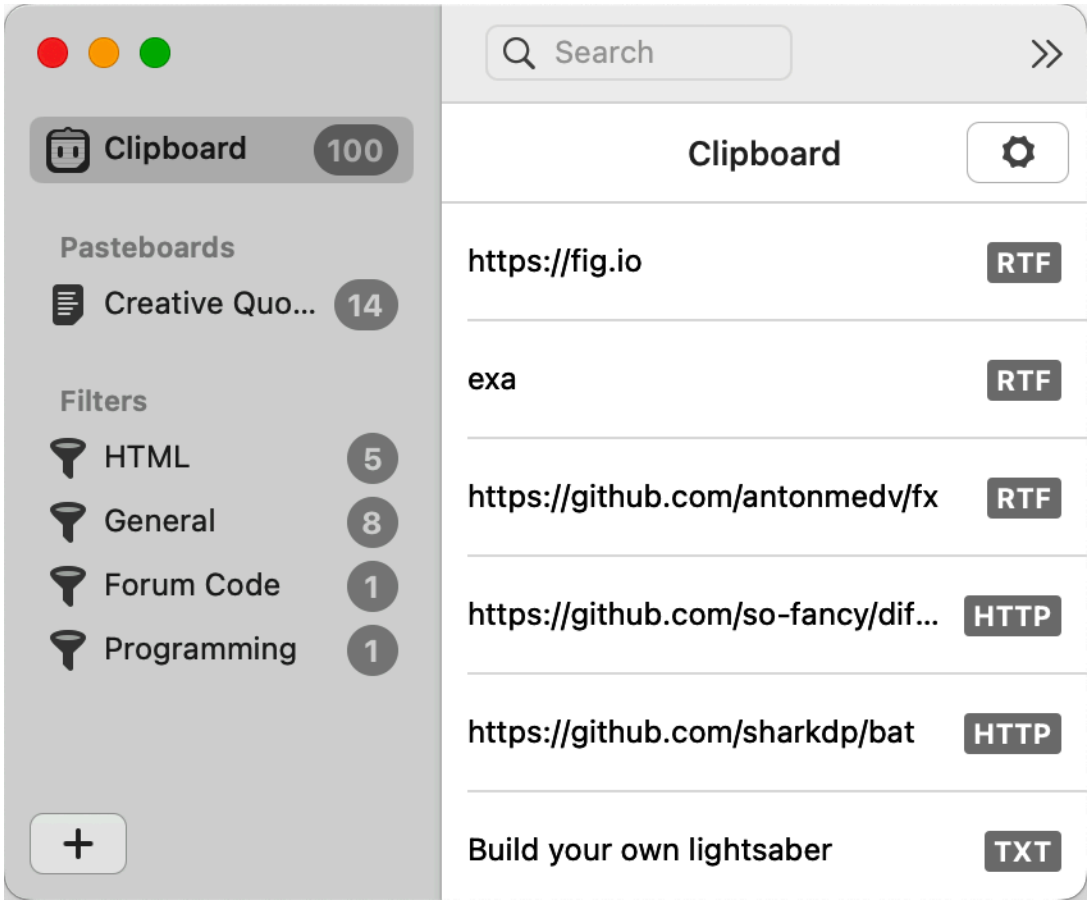



Figura 11-1. Pastebot te da acceso al historial del portapapeles

Los expansores de texto son otra excelente herramienta de productividad. Las herramientas independientes como TextExpander pueden aumentar enormemente la productividad. Muchos sistemas operativos también tienen una función de sustitución de texto integrada (véase la

figura 11-2) que tú puedes configurar completamente. Una vez más, los pequeños ahorros se acumulan con el tiempo.

Como puedes ver, tu entorno está repleto de opciones de personalización. No te conformes con los valores predeterminados: no debes adaptarte a tus herramientas, sino que ellas deben adaptarse a ti. El tiempo que dedicas a ajustar tu entorno es una inversión en tu productividad.

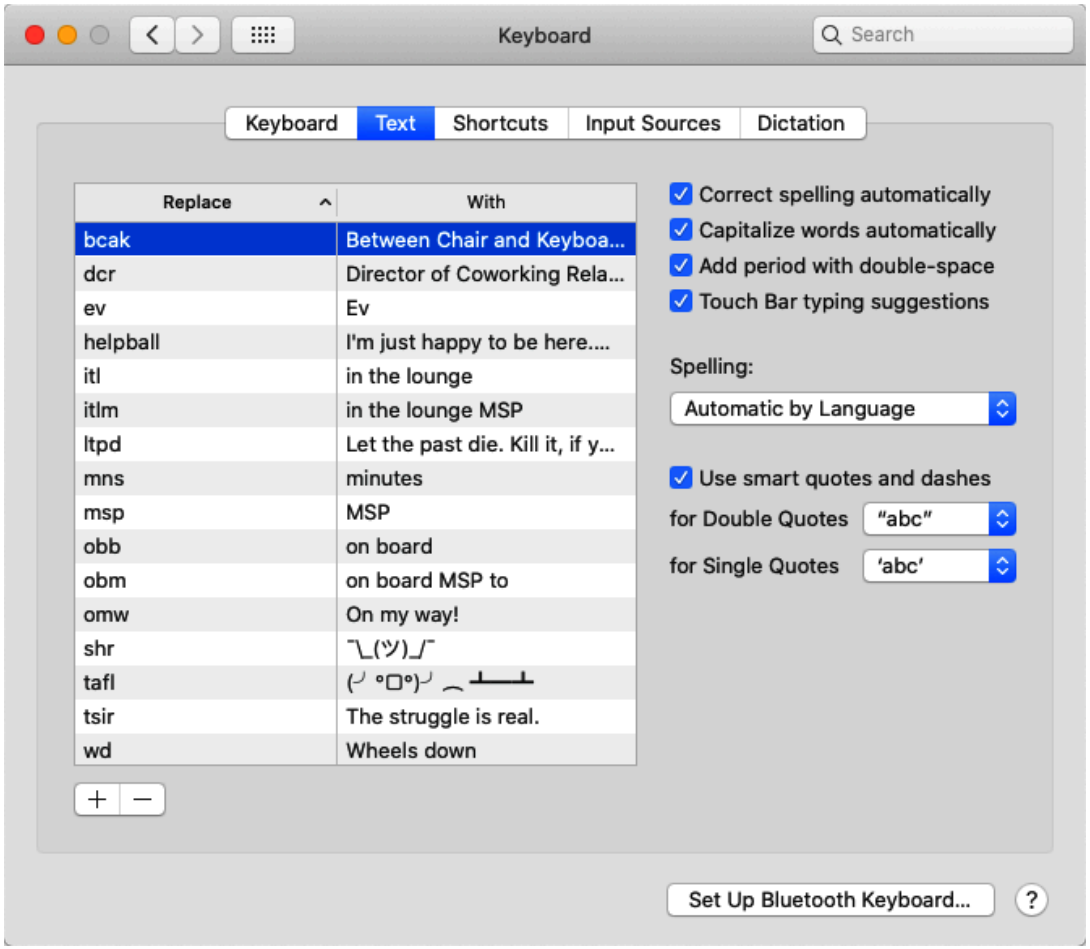



Figura 11-2. Las opciones de sustituciónde texto pueden mejorar tu productividad

# Automatización estratégica

Tú, desarrollador, pasas todo el día escribiendo código para otros, así que no tengas miedo de escribir algo para ayudarte a ti mismo. Unas pocas líneas de código podrían ahorrarte horas y horas de trabajo, y también te proporcionan una excusa realmente buena para jugar con otros lenguajes y técnicas. Muchos desarrolladores tienen una relación de amor-odio con las expresiones regulares, pero aprender aunque sea un poco sobre regex puede ser increíblemente útil.

Nunca olvides que, en informática, solo hay tres números. Hay cosas que haces cero veces, cosas que haces una vez y solo una vez, y luego *n*. Si haces algo más de una vez, debes esperar hacerlo innumerables veces. Debes considerar dedicar tiempo a automatizar cualquier cosa que hagas más de una vez.

---

**ADVERTENCIA**

La búsqueda de la automatización puede ser una peligrosa pérdida de tiempo. A veces, escribir el código puede llevar más tiempo que la propia tarea. Si una tarea es realmente puntual y es poco probable que se repita, automatizarla no es una buena inversión de tiempo. Considera la posibilidad de esperar hasta que hayas hecho algo varias veces antes de intentar automatizarlo, e incluso entonces, asegúrate de limitar el tiempo que le dedicas. Al final, utiliza tu mejor criterio.

---

Para ello, escribe scripts auxiliares e . Treinta líneas de Python podrían ahorrarte horas de trabajo. La moraleja de la historia es que la automatización es tu amiga; deja que el ordenador haga más trabajo por ti. Y si no te apetece escribir el script tú mismo, ¡siempre puedes dejar que la IA lo escriba por ti! Aún así, debes leer y probar el código, pero la barrera para la automatización es más baja que nunca.

# La búsqueda perpetua de hábitos productivos

La gran cantidad de atajos, utilidades y comandos es prácticamente infinita, por lo que aprenderlos puede parecer abrumador. No intentes aprenderlos todos en uno o dos meses; es un proceso continuo. El objetivo es mejorar un poco cada día; intenta aprender algo nuevo cada semana. Este método es más accesible y sostenible, y también hace que sea más probable que retengas lo que estás aprendiendo.

## Aprendizaje colaborativo

Una de las mejores fuentes de consejos y trucos son tus compañeros de trabajo. Acostúmbrate a preguntar a tus compañeros de equipo sobre sus prácticas de productividad. Si un colega utiliza una herramienta, un comando o un atajo que no has visto antes, pregúntale al respecto. No te sientas mal por no estar familiarizado con ello. Del mismo modo, si ves a un compañero de equipo haciendo algo de la manera difícil, tómate el tiempo para mostrarle el enfoque más sencillo. Para reforzar el concepto, tómate el tiempo de repetirlo tres o cuatro veces antes de

continuar. Si tu compañero de trabajo no sabe cómo empezar a utilizar una de tus herramientas favoritas, ayúdale a ponerse en marcha. Al hacerlo, no solo afianzarás tus conocimientos, sino que también podrás disponer de un entorno familiar.[7]

Dedica unos minutos cada semana a aprender algo nuevo sobre tus herramientas y considera la posibilidad de compartir lo que has aprendido con tus amigos y compañeros de trabajo. Si estás haciendo programación en par e y tu compañero está utilizando una herramienta o utilidad que no conoces, pídele que te la enseñe. Si tu compañero no utiliza un atajo que tú conoces, enséñale a utilizarlo. Considera la posibilidad de organizar una sesión semirregular de almuerzo y aprendizaje como foro para que la gente comparta cosas interesantes que utiliza; no des por sentado que todo el mundo sabe lo mismo que tú. Probablemente no sea así. Del mismo modo, no te saltes esa sesión o seminario web sobre una herramienta que conoces bien; no des por sentado que no hay nada nuevo que aprender.

Este enfoque se extiende al lenguaje que elijas. Los lenguajes evolucionan,[8] por lo que tu código también debe hacerlo. Cuando salga una nueva versión, estudia las notas de la versión y tómate tu tiempo para explorar las nuevas funciones. Enseña a los demás lo que has aprendido y estate atento a las oportunidades para poner en práctica esos nuevos enfoques. Dirige un grupo de estudio: el enfoque «ver uno, hacer uno, enseñar uno» es eficaz.

## Gestión del conocimiento personal

La tecnología cambia a un ritmo increíblemente rápido, lo que significa que te verás inundado de información. Desde podcasts hasta entradas de blog, pasando por vídeos y tutoriales, te conviene tener una estrategia de gestión del conocimiento personal. No tiene por qué ser compleja ni implicar herramientas específicas, pero debes tener un repositorio donde puedas almacenar información interesante para su uso posterior. Debes estar preparado para capturar y organizar el flujo de información en tu mundo. No hay nada más frustrante que *saber* que tenías la respuesta exacta a una pregunta, pero no recordar dónde está.

Toma notas. *Volverás a encontrarte* con ese problema. Y también lo hará un compañero de equipo. Anótalo. No lo recordarás la próxima vez. Descárgalo de tu cerebro: el almacenamiento externo es la mejor opción. No importa cómo, no importa si es formal. ¿Un clip con trozos de papel al azar? Eso también funciona, aunque las variantes electrónicas son sin duda más fáciles de buscar, y el papel puede ser más difícil de compartir con los compañeros.

Algunos desarrolladores guardan una nota o un documento en su ordenador portátil. Otros prefieren un simple cuaderno como los de Field Notes o Moleskine, junto con un bolígrafo de bolsillo como el Mark Two o el Space Pen. Muchos optan por una solución más técnica como Org Mode, Evernote o Notion. Piensa también en cómo quieres gestionar los diversos recursos web que descubrirás a lo largo del día. Sincronizar tus marcadores (organizados) entre dispositivos es una op-

ción, pero también puedes utilizar gestores de marcadores específicos como Pinboard.

NOTION

Aquí Dan. Como ingeniero de software y amante del orden, he probado numerosas herramientas de gestión del conocimiento, y Notion es mi favorita. Me encanta porque me da la flexibilidad de usarla tanto o tan poco como necesite. Estas son solo algunas de las formas en que utilizo Notion como desarrollador de software:

*Aprendizaje*

Cuando aprendo un nuevo idioma, marco de trabajo o herramienta, consolido toda mi investigación en Notion. Al combinar notas, fragmentos de código, imágenes y recursos, acelero mi proceso de aprendizaje.

*Documentación*

Para documentar nuevos procesos o funciones, empiezo en Notion. Su editor de formato libre es compatible con Markdown y ofrece una vista previa en tiempo real, lo que mejora mi flujo de trabajo de documentación.

*Listas*

Almaceno varias listas de verificación en Notion, como mi lista de verificación de revisión de código. Me ayuda a asegurarme de que cubro todos los puntos necesarios durante una revisión.

*Toma de notas*

Notion me sirve como centro neurálgico para mis notas, ya que me permite estructurarlas de la forma que mejor me conviene. La aplicación móvil me permite acceder a mis notas desde cualquier lugar, mientras que la extensión para el navegador

Notion Clipper me ayuda a guardar elementos para consultarlos más adelante.

El verdadero poder de Notion reside en su versatilidad y personalización. Me permite crear un espacio de trabajo personalizado que se adapta a mis necesidades y flujos de trabajo únicos. Notion es mi segundo cerebro, y no podría imaginar mi vida como ingeniero sin él.

Dicho esto, no hay escasez de herramientas en este ámbito y, aunque Notion es una parte vital de mi flujo de trabajo, tu experiencia puede variar. Debes utilizar lo que mejor te funcione.

---

Asistir a conferencias es una forma inestimable de aprender nuevas habilidades y perfeccionar las ya existentes. Pero querrás capturar lo que has aprendido para estudiarlo en el futuro. Muchas conferencias publican vídeos de las presentaciones después del evento. Tómate tu tiempo para identificar las charlas que más te interesan y crea tu propia lista de reproducción de vídeos para reproducirlos o recordarlos más fácilmente más adelante. Algunos desarrolladores incluso tratan estas charlas como un podcast y las ponen en cola para escucharlas cuando salen a pasear, hacen ejercicio o se desplazan al trabajo. Incluso puedes escucharlas a una velocidad superior a 1,0, lo que te permite consumir más contenido más rápidamente.

Sea cual sea el enfoque que elijas, asegúrate de que no esté limitado a un solo dispositivo; debe ser portátil y funcionar en un contexto móvil. Nunca se sabe cuándo te llegará la inspiración y te dará una idea clave para resolver un problema en el trabajo, así que prepárate para captu-

rarla. Nunca se sabe cuándo un amigo te recomendará un libro o un podcast increíble, así que prepárate para capturarlo.

---

**MANTÉN LA SENCILLEZ**

Aquí Nate. No hay nada de malo en la simplicidad. Al principio de mi carrera, tenía un compañero que tenía un montón de trozos de papel unidos con un clip. Había garabateado varios comandos, consejos, instrucciones y otras cosas efímeras que había recopilado a lo largo de los años. Cuando le hacías una pregunta, era muy probable que hojease sus notas y encontrase la información exacta que necesitabas. Aunque tenía un sistema de gestión del conocimiento sencillo, le funcionaba, y eso es lo que importa.

---

# Conclusión

Un buen ingeniero de software controla su kit de herramientas. Del mismo modo que dedicarías unos minutos a ajustar el asiento, los espejos, el volante y la temperatura en un coche nuevo, tu entorno de desarrollo debe adaptarse a ti y a tus necesidades. No te limites a aceptar la configuración predeterminada. Personaliza tu editor y tu sistema operativo. Los usuarios avanzados utilizan herramientas avanzadas: los atajos de teclado y la línea de comandos pueden multiplicar tu productividad. Te sorprendería saber lo mucho que una utilidad gratuita o de bajo coste puede cambiar tu día a día, así que no tengas miedo de probar cosas nuevas.

Llevar un registro de todos estos atajos y utilidades puede resultar abrumador, así que no confíes en tu memoria. Perfecciona tus habilidades de gestión del conocimiento a medida que amplías tu conjunto de herramientas. Aprende de tus compañeros y enséñales.

## Ponerlo en práctica

Experimenta con nuevas formas de ser más productivo, pero ten cuidado de que tus experimentos no te hagan perder tiempo. Elige una tarea repetitiva y comprueba si puedes automatizarla para ahorrarte el esfuerzo. Considera la posibilidad de desarrollar una automatización existente para cubrir algo más significativo. Comparte lo que aprendas con tus colegas.

Comprométete a aprender un nuevo atajo de teclado a la semana durante los próximos meses.

Dedica 15 minutos los viernes por la tarde a leer detenidamente la documentación de tu editor favorito. ¿Qué has aprendido que no sabías antes? Activa la función «consejo del día» de tu editor o terminal: ¿qué función te ha sorprendido?

Examina tu enfoque de gestión del conocimiento: ¿te está funcionando? Si no es así, prueba algo diferente. No tiene por qué ser complejo ni implicar herramientas sofisticadas; solo tiene que funcionar para ti.

Cuando aprendas un nuevo comando, ¡anótalo! Aprovecha un cerebro externo.

# Recursos adicionales

- *Flow: The Psychology of Optimal Experience*, de Mihaly Csikszentmihalyi (Harper & Row, 1990)
- *The Productive Programmer*, de Neal Ford (O'Reilly, 2008)
- *Construye un segundo cerebro: un método probado para organizar tu vida digital y liberar tu potencial creativo*, de Tiago Forte (Simon Element / Simon Acumen, 2022)
- *The Passionate Programmer*, 2.ª edición, de Chad Fowler (Pragmatic Press, 2009)

1 Una vez que hayas encontrado un tema estupendo, es posible que quieras cambiar una o tres cosas, no tienes por qué conformarte con la configuración predeterminada.

2 Si te encuentras con dolores y molestias extraños, el alivio podría ser tan simple como cambiar de estar sentado a estar de pie a lo largo del día.

3 Aunque el precio de una silla y un escritorio de alta gama pueda parecerte excesivo, los equipos de calidad duran mucho tiempo.

4 No esperes a sufrir una lesión por esfuerzo repetitivo. Si tienes problemas, un teclado mejor puede hacer maravillas.

5 *La fila central es* la fila de teclas en la que descansan naturalmente los dedos al escribir a máquina, y la mayoría de los teclados incluyen un marcador táctil para indicar dónde deben descansar los dedos índices.

6 La tecla Meta suele estar asignada a Alt, Bloq Mayús o Escape, dependiendo de tu sistema operativo.

7 Los equipos pueden decidir un nivel básico de coherencia en las herramientas para garantizar la facilidad de emparejamiento.

8 Muchos lenguajes y marcos fomentan las contribuciones de la comunidad. No tengas miedo de dar forma al futuro de tu entorno participando en él.